

is the opposite of whether 41 is odd or even. The function will return `not isOdd(41)`. This function call, in turn, returns the opposite Boolean value of `isOdd(40)`, and so on, until `isOdd(0)` returns `False`. The number of recursive function calls determines the number of `not` operators that act on return values before the final return value is returned.

However, this recursive function results in stack overflows for large-number arguments. Calling `isOdd(100000)` results in 100,001 function calls without returning—which far exceeds the capacity of any call stack. We can rearrange the code in the function so that the last action of the recursive case is returning the results of the recursive function call, making the function tail recursive. We do this in `isOddTailCall()` in *isOdd.py*. Here is the rest of the *isOdd.py* program:

Python

```
--snip--
def isOddTailCall(number, inversionAccum=False):
    if number == 0:
        # BASE CASE
        return inversionAccum
    else:
        # RECURSIVE CASE
        return isOddTailCall(number - 1, not inversionAccum)

print(isOddTailCall(42))
print(isOddTailCall(99))
```

The JavaScript equivalent is in the rest of *isOdd.html*:

JavaScript

```
--snip--
function isOddTailCall(number, inversionAccum) {
    if (inversionAccum === undefined) {
        inversionAccum = false;
    }

    if (number === 0) {
        // BASE CASE
        return inversionAccum;
    } else {
```